

Graph-Based Intrusion Detection: Detecting Network Attacks with Graph Neural Networks and LLM-Generated Alert Explanations

Shiyi Liu (Team Leader)
Dept. of Computer Science
Virginia Tech
lshiyi25@vt.edu

Vighnesh Nair
Dept. of Computer Science
Virginia Tech
vighneshnair@vt.edu

Krishna Inukonda
Dept. of Computer Science
Virginia Tech
kinukond@vt.edu

Gokulakrishnan Ravi
Dept. of Computer Science
Virginia Tech
gravi@vt.edu

Abstract—Network intrusion detection systems (NIDS) based on machine learning typically classify each network flow in isolation, treating it as an independent feature vector. This flow-level view discards the communication structure of the network, which is precisely where many attacks become visible: a port scan, a distributed denial-of-service flood, or lateral movement is defined by its pattern across many hosts rather than by any single connection. This project proposes a graph-based intrusion detection framework in which hosts are modeled as nodes and network flows as attributed edges, and attack detection is formulated as an edge classification task solved with a graph neural network (GNN). We plan to construct time-windowed flow graphs from the standardized NetFlow datasets NF-UNSW-NB15-v2 and NF-CSE-CIC-IDS2018-v2, implement an edge-aware GNN based on E-GraphSAGE, and compare it against flow-level baselines including Random Forest, XGBoost, and a multilayer perceptron under consistent train/validation/test splits designed to minimize data leakage. Because the two datasets share a common feature schema, we plan to evaluate cross-dataset generalization using a binary benign-versus-malicious formulation, training on one network and testing on the other. This design avoids inconsistencies between the attack taxonomies of the two datasets while still allowing us to measure cross-network transfer. Finally, as an extension to the core detection pipeline, we plan to add a large language model (LLM) explanation layer that converts flagged flows and their local graph context into human-readable alerts, with the results presented through an interactive dashboard. Evaluation will emphasize macro and per-class F1 rather than accuracy, reflecting the severe class imbalance inherent to intrusion data.

Index Terms—intrusion detection, graph neural networks, network security, deep learning, edge classification, domain generalization, large language models

I. INTRODUCTION

A. Motivation

Enterprise and campus networks are subject to continuous automated attack traffic, and the volume of telemetry produced by modern networks makes manual inspection infeasible. Machine-learning-based NIDS have therefore become a standard component of defensive infrastructure. The dominant formulation treats each network flow—a summarized record of a connection between two endpoints—as an independent sample described by features such as duration, byte counts, packet

counts, protocol, and port numbers, and trains a classifier to label it as benign or malicious.

This formulation has a structural blind spot. Many attack classes are not identifiable from a single flow at all. A single SSH connection is unremarkable; three hundred SSH connections from one host to three hundred distinct destinations within two minutes is a brute-force sweep. A single HTTP request is unremarkable; ten thousand requests converging on one destination from many sources is a denial-of-service attack. In both cases the evidence resides in the *topology* of communication rather than in the attributes of any one connection. Flow-level classifiers can only approximate this evidence through hand-engineered aggregate features, which are brittle and must be redesigned for each new attack pattern.

B. Problem Statement

We address the problem of classifying individual network flows as benign or as belonging to a specific attack class, while explicitly exploiting the graph structure of host-to-host communication. Formally, given a time window t , we construct a directed multigraph $G_t = (V_t, E_t)$ in which V_t is the set of hosts observed in the window and each edge $e \in E_t$ corresponds to an observed flow carrying a feature vector $\mathbf{x}_e \in \mathbb{R}^d$. The task is to learn a function $f : e \mapsto y_e$ mapping each edge to a label $y_e \in \{\text{benign}, c_1, \dots, c_k\}$, where c_i are attack categories.

Three difficulties make this non-trivial. First, standard GNNs propagate information over node features, whereas here the informative signal lives on edges, requiring an edge-centric message-passing formulation. Second, attack classes are severely imbalanced, with some categories representing a fraction of a percent of all flows, so accuracy is an uninformative metric and naive training collapses to the majority class. Third, published NIDS results are often inflated by evaluation artifacts: random train/test splits allow the model to memorize host identifiers, and duplicate records inflate apparent performance. A credible evaluation must control for all three.

C. Significance

If the graph-based detector outperforms flow-level baselines, this would provide evidence that communication structure contributes useful detection signal beyond per-flow features alone. The cross-dataset experiment is intended to evaluate how well the learned representations transfer across different network environments. Measuring the performance drop across datasets provides an indication of the model’s robustness to distribution shift and helps assess the practical limitations of cross-network generalization. The LLM explanation layer addresses a complementary interpretability challenge: numeric class labels alone may provide limited context for analysts, while natural-language explanations can make model outputs easier to inspect and understand. Converting a prediction together with its local graph context into a short natural-language explanation provides a concrete output that can be qualitatively evaluated for factual consistency and clarity.

II. RELATED WORK

A. Machine Learning for Intrusion Detection

Intrusion detection has historically been divided into signature-based and anomaly-based approaches. Signature-based systems match traffic against a database of known attack patterns and are precise but unable to detect previously unseen attacks, since a signature must be authored before the attack can be recognized. Anomaly-based systems instead model normal behavior and flag deviations, which offers the possibility of detecting novel attacks at the cost of a higher false positive rate. Machine learning has been applied predominantly to the second category, and more recently to supervised multi-class attack categorization.

The evolution of this line of work is closely tied to the evolution of its benchmark datasets. Early research relied heavily on KDD Cup 1999 and its cleaned successor NSL-KDD [9], which removed the large number of duplicate records that allowed classifiers to achieve misleadingly high scores. These datasets are now widely regarded as unrepresentative of modern traffic, having been superseded by UNSW-NB15 [7], which combines real benign traffic with synthesized contemporary attacks, and by the CIC series culminating in CSE-CIC-IDS2018 [8], which captures a large-scale testbed with a broad attack taxonomy.

On these flow-level feature sets, classical ensemble methods remain strong. Random Forest and gradient-boosted trees such as XGBoost consistently perform competitively on tabular network features, and are frequently reported to match or exceed deep models in the in-domain setting. Deep learning approaches have nevertheless been explored extensively: autoencoders for unsupervised anomaly scoring, convolutional networks applied to flow features reshaped into image-like grids, and recurrent architectures applied to sequences of packets or flows.

What unites essentially all of this work is the independence assumption. Whether the classifier is a decision tree ensemble or a deep network, each flow is scored on its own feature

vector. Relational information between flows enters only indirectly, through aggregate features such as connection counts within a fixed time window, which must be specified by hand and generalize poorly to attack patterns the designer did not anticipate.

B. Graph Neural Networks

Graph neural networks generalize deep learning to data with explicit relational structure. The dominant formulation is message passing: each node iteratively updates its representation by aggregating representations from its neighbors, so that after K rounds a node’s embedding summarizes its K -hop neighborhood. Graph convolutional networks [4] established this framework with a normalized averaging aggregator, but operate transductively, requiring the full graph at training time and providing no principled way to embed nodes that were not present during training.

GraphSAGE [3] addressed this limitation by learning aggregator functions rather than node-specific embeddings, and by sampling a fixed-size neighborhood rather than using all neighbors. This makes the method inductive: a trained model can produce embeddings for nodes it has never seen. That property is essential in our setting, where the hosts appearing in deployment are not the hosts seen during training, and a transductive model would be unusable.

Graph attention networks [5] replace uniform or degree-normalized aggregation with learned attention coefficients, so that a node can weight its neighbors according to their relevance. In an intrusion detection context this is an attractive inductive bias: among the many connections a busy server participates in, only a small subset may be attack-related, and a mean aggregator dilutes that signal across all neighbors while attention can concentrate on it.

A practical difficulty for our task is that most GNN literature assumes the information of interest resides on *nodes*, with edges serving only as connectivity. In network traffic the situation is inverted: the rich feature vector belongs to the flow, that is, the edge, while hosts carry little intrinsic information. Applying GNNs to intrusion detection therefore requires an edge-centric reformulation of message passing rather than an off-the-shelf architecture.

C. Graph-Based Network Intrusion Detection

A growing body of work applies graph learning to network security by representing traffic as a communication graph in which hosts are nodes and flows are edges. This representation is a natural fit for the attack classes described in Section I, since scanning, flooding, and lateral movement all produce characteristic topological signatures such as high fan-out, high fan-in, or unusual chains of internal connections.

E-GraphSAGE [1] is the closest prior work to this proposal and serves as our starting point. It adapts GraphSAGE so that node representations aggregate the features of *incident edges* rather than neighboring nodes, then classifies each edge from the concatenation of its endpoint embeddings. Node features are initialized as constant vectors, which is a deliberate design

choice: it prevents the model from using IP addresses as predictive features and forces it to rely on flow attributes and topology. The method was shown to outperform flow-level baselines on several NetFlow-format IoT intrusion datasets.

Related efforts extend this direction in different ways. Anomal-E [2] pursues a self-supervised variant, learning edge embeddings without attack labels and then applying anomaly detection to those embeddings, which removes dependence on labeled attack data at the cost of coarser attack categorization. Other work constructs host-level communication graphs over longer horizons to target slow, multi-stage intrusions rather than individual flows.

Two limitations recur across this literature. First, evaluation is usually confined to a single dataset, so the reported numbers say little about whether the learned structural patterns transfer to a different network. Second, graph construction choices—window length, directional treatment, handling of high-degree hubs—are often stated without ablation, making it unclear how much of the reported gain follows from the architecture as opposed to the particular graph built from the data.

D. Evaluation Pitfalls and Generalization

Reported NIDS performance is unusually sensitive to experimental protocol, and a substantial body of criticism documents how easily results become inflated. The best-known case is the duplicate-record problem in KDD Cup 1999, which biased classifiers toward frequently repeated records and motivated the construction of NSL-KDD [9]; comparable duplication exists in more recent datasets and requires explicit removal.

A second and more subtle issue concerns splitting. Random train/test partitioning places flows from the same attack episode on both sides of the split, so a model can achieve high test scores by recognizing episode-specific artifacts rather than attack behavior. Identifier leakage compounds this: when source and destination IP addresses are retained as features, a classifier can simply memorize which addresses belong to the attacking machines in the capture, producing near-perfect scores that collapse entirely on any other network. Temporal or episode-aware splitting combined with removal of identifier fields is a common mitigation strategy, and we plan to incorporate both where supported by the dataset structure.

A third issue is metric selection. Benign traffic dominates every realistic capture, so a classifier that labels everything benign can report very high accuracy. Macro-averaged and per-class F1 are therefore the appropriate primary metrics, as they expose failure on rare attack categories that aggregate accuracy conceals.

Cross-dataset evaluation has historically been obstructed by the fact that each dataset ships its own feature extraction pipeline and feature set, making a model trained on one incompatible with another. The standardized NetFlow feature schema of Sarhan et al. [6] removes this obstacle by re-processing several widely used intrusion datasets into a single common format. This makes train-on-one, test-on-another experiments feasible without feature re-engineering, yet such experiments

remain comparatively rare, and the performance drops reported when they are conducted are often substantial.

E. Summary and Gap

Prior work establishes that GNNs are applicable to flow classification and that standardized feature schemas make cross-dataset comparison possible, but these directions are not commonly evaluated together. Many graph-based NIDS studies focus on a single dataset, and natural-language alert explanation is not typically integrated into the graph-based detection pipeline. Our project therefore focuses on combining edge-centric graph learning, leakage-aware evaluation, cross-dataset generalization, and human-readable alert explanation within a unified framework.

III. DATASET DESCRIPTION

A. Source Datasets

We use two datasets from the standardized NetFlow NIDS collection [6]:

- **NF-UNSW-NB15-v2**, derived from the UNSW-NB15 testbed capture [7], containing attack categories including Exploits, Fuzzers, Reconnaissance, DoS, Generic, Analysis, Backdoor, Shellcode, and Worms.
- **NF-CSE-CIC-IDS2018-v2**, derived from CSE-CIC-IDS2018 [8], containing categories including DoS, DDoS, Brute Force, Botnet, Infiltration, and Web Attacks.

Both are expressed in an identical extended NetFlow schema, which is the property that makes a train-on-one, test-on-the-other experiment possible without any feature re-engineering.

B. Record Fields

Each flow record provides:

- Source and destination IPv4 addresses and ports.
- Layer-4 protocol and Layer-7 protocol identifier.
- Byte and packet counts in both directions.
- Flow duration and inter-arrival time statistics.
- TCP flag aggregates and TTL statistics.
- A binary label and a multi-class attack category.

C. Graph Construction

Flows are partitioned into fixed-length time windows of 60 seconds. Within each window, every distinct IP address becomes a node and every flow becomes a directed edge carrying its full feature vector. Because host identity must not be used as a predictive feature, node features are initialized as constant vectors rather than derived from IP addresses, forcing the model to rely on edge attributes and local topology.

D. Preprocessing

- Removal of exact duplicate records.
- Removal of direct host identifiers such as source and destination IP addresses from the predictive feature set to reduce identifier leakage. Port-related features are retained unless they are found to introduce dataset-specific artifacts.

- Log transformation of selected heavy-tailed count features, followed by standardization using statistics fitted on the training data only.
- Categorical protocol-related features are encoded consistently across the training, validation, and test sets.
- Train/validation/test splitting designed to minimize temporal and episode-level leakage. Where reliable timestamps are available, flows are ordered chronologically before splitting so that future traffic does not inform model training.

E. Class Imbalance

Both datasets exhibit substantial class imbalance, with benign traffic representing a large share of the observations and some attack categories occurring much less frequently. We plan to address this using class-weighted loss during model training and to report macro and per-class F1 scores so that performance on minority attack classes is not obscured by aggregate metrics.

F. Limitations

- Both datasets were collected in controlled testbed environments and may not fully represent the traffic distributions of real production networks.
- Attack labels are derived from the experimental capture scenarios and therefore depend on the assumptions and design of the original datasets.
- NetFlow representations summarize connection-level behavior and do not include packet payload content, so payload-based attack signatures are outside the scope of this work.
- Fixed-length time-window graph construction introduces artificial boundaries, which may split attack activity that spans multiple windows.

IV. PROPOSED APPROACH

A. Baselines

To evaluate whether graph structure provides additional predictive value beyond flow-level features, we plan to compare the proposed GNN against three non-graph baselines trained using the same input features, data splits, and class-weighting strategy:

- 1) Random Forest.
- 2) XGBoost.
- 3) A multilayer perceptron (MLP) with a parameter count chosen to be reasonably comparable to the GNN.

B. Graph Neural Network Model

Our primary graph model is based on the E-GraphSAGE formulation, which is designed for settings in which informative features are associated primarily with edges rather than nodes. At each message-passing layer, a node updates its representation by aggregating information from the incident network-flow edges:

$$h_v^{(k)} = \sigma \left(W^{(k)} \cdot \text{CONCAT} \left(h_v^{(k-1)}, \text{AGG}_{u \in N(v)} \left(h_{uv}^{(k-1)} \right) \right) \right) \quad (1)$$

where h_{uv} denotes the representation associated with the flow edge between nodes u and v , and AGG denotes a permutation-invariant aggregation function such as mean aggregation.

After K message-passing layers, the representation used for edge classification is constructed by concatenating the learned embeddings of the source and destination nodes with the original flow features:

$$z_{uv} = \text{CONCAT} \left(h_u^{(K)}, h_v^{(K)}, x_{uv} \right) \quad (2)$$

The resulting edge representation is then passed to a classification head that outputs a probability distribution over the benign class and the available attack categories.

C. Proposed Extension

Beyond the baseline E-GraphSAGE model, we plan to investigate an attention-based aggregation mechanism. Instead of assigning equal importance to all neighboring flows during message passing, the attention mechanism learns weights that allow the model to emphasize more informative or suspicious connections.

This extension is motivated by the observation that network hosts may participate in many benign connections while only a small subset of their neighboring flows are associated with malicious behavior. Mean aggregation may dilute these important signals, whereas attention-based aggregation may allow the model to focus more strongly on the most relevant communication patterns.

We will compare attention-based aggregation against the original mean aggregation used in the baseline model to evaluate whether learned neighbor weighting improves intrusion detection performance, particularly for minority attack classes.

D. LLM Explanation Layer

As an extension to the core intrusion detection pipeline, we plan to add a large language model (LLM) explanation layer that converts model predictions into short, human-readable security alerts. The LLM will not participate in the detection decision itself; instead, it will receive structured information derived from the GNN output and the corresponding local graph context.

For each flow predicted as malicious, the explanation input will include the predicted attack class, prediction confidence, selected flow features, and local graph statistics such as source-host fan-out, the number of distinct destination ports contacted, and the number of similar flows observed within the same time window. This information will be passed to an LLM using a fixed prompt template through the Virginia Tech ARC LLM Gateway.

The generated explanation will summarize why the flow was flagged in plain language while remaining grounded in the supplied model and traffic evidence. To reduce unsupported

explanations, the LLM will be instructed to use only the information provided in the structured input and not introduce additional attack evidence. The explanation component will be evaluated qualitatively for factual consistency and clarity.

E. Evaluation Protocol

- **Primary metrics:** macro F1 and per-class F1.
- **Secondary metrics:** precision, recall, confusion matrices, and precision–recall AUC where appropriate.
- **In-domain evaluation:** held-out train/validation/test splits within each dataset, using chronological splitting where reliable timestamps are available.
- **Cross-dataset evaluation:** train on NF-UNSW-NB15-v2 and test on NF-CSE-CIC-IDS2018-v2, and vice versa, using a binary benign-versus-malicious formulation to avoid inconsistencies between the attack taxonomies of the two datasets.
- **Ablation studies:** we prioritize two comparisons: graph-based modeling versus flow-level modeling, and attention-based aggregation versus mean aggregation. Additional studies on time-window length and class weighting will be conducted if time permits.
- **Qualitative evaluation:** manual review of a sample of LLM explanations for factual consistency with the underlying flow.

V. SYSTEM DESIGN

A. Architecture Overview

The system is organized as a four-stage pipeline: data ingestion and graph construction, model training, inference and explanation, and visualization.

B. Data Layer

The data layer is responsible for loading the standardized NetFlow datasets, removing duplicate records, preprocessing flow features, and constructing time-windowed communication graphs. Source and destination IP addresses are used only to define graph connectivity and are excluded from the predictive feature set to reduce identifier leakage.

Within each time window, each distinct host is represented as a node and each network flow is represented as a directed edge containing the corresponding flow features. The resulting graph objects are serialized to disk so that graph construction does not need to be repeated for every training run.

All preprocessing parameters, including feature scaling statistics and categorical encodings, are fitted using the training data and then applied consistently to the validation and test sets.

C. Model Layer

Models will be implemented in PyTorch with PyTorch Geometric. Training runs will execute on the Virginia Tech CS GPU cluster (glogin.cs.vt.edu). All runs will be configuration-driven and seeded, with metrics logged per epoch to support reproducibility.

D. Explanation Layer

The explanation module receives malicious-flow predictions from the trained GNN together with structured traffic and graph-context information. For each flagged flow, the module prepares a compact summary containing the predicted class, confidence score, selected flow features, and local graph statistics.

This structured summary is sent to an LLM through the Virginia Tech ARC LLM Gateway using a fixed prompt template. The LLM is instructed to generate a short analyst-facing explanation using only the supplied evidence and not to introduce unsupported information.

Generated responses are cached so that repeated views of the same alert do not require additional LLM calls. This layer is used only for explanation and does not influence the intrusion detection decision.

E. Visualization Layer

If time permits, we will develop a lightweight interactive dashboard to present model predictions and evaluation results. The dashboard will allow users to inspect a selected time window, view the corresponding communication graph, and identify flows predicted as malicious.

For a selected malicious flow, the interface will display the predicted attack class, confidence score, relevant traffic and graph-context information, and the corresponding LLM-generated explanation. Additional views will summarize model performance through confusion matrices, per-class F1 scores, and comparisons between the GNN and non-graph baselines.

The dashboard is intended primarily as a demonstration and analysis interface rather than as a production-ready monitoring system.

F. Repository Structure

data/	raw and processed datasets
src/graph/	graph construction
src/models/	GNN and baseline models
src/eval/	metrics and evaluation
src/llm/	explanation generation
dashboard/	visualization
configs/	experiment configurations

REFERENCES

- [1] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, “E-GraphSAGE: A graph neural network based intrusion detection system for IoT,” in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2022.
- [2] E. Caville, W. W. Lo, S. Layeghy, and M. Portmann, “Anomal-E: A self-supervised network intrusion detection system based on graph neural networks,” *Knowledge-Based Systems*, 2022.
- [3] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [4] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. International Conference on Learning Representations (ICLR)*, 2017.
- [5] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. International Conference on Learning Representations (ICLR)*, 2018.

- [6] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, “NetFlow datasets for machine learning-based network intrusion detection systems,” in *Big Data Technologies and Applications*, Springer, 2021.
- [7] N. Moustafa and J. Slay, “UNSW-NB15: A comprehensive data set for network intrusion detection systems,” in *Proc. Military Communications and Information Systems Conference (MilCIS)*, 2015.
- [8] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.
- [9] M. Tavallaei, E. Bagheri, W. Lu, and A. A. Ghorbani, “A detailed analysis of the KDD CUP 99 data set,” in *Proc. IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, 2009.

APPENDIX A TASK ASSIGNMENT FOR EACH MEMBER

TABLE I
TASK ASSIGNMENT

Task	Assigned To
Dataset acquisition and duplicate removal	Shiyi Liu
Feature preprocessing and normalization	Shiyi Liu
Temporal train/validation/test splitting	Shiyi Liu
Time-windowed graph construction	Shiyi Liu
Random Forest and XGBoost baselines	Vighnesh Nair
MLP baseline	Vighnesh Nair
Shared evaluation module (macro/per-class F1)	Vighnesh Nair
Ablation experiments	Krishna Inukonda
E-GraphSAGE implementation	Krishna Inukonda
Proposed model extension	Krishna Inukonda
Hyperparameter tuning and class weighting	Krishna Inukonda
Cross-dataset generalization experiments	Krishna Inukonda
LLM prompt design and explanation module	Gokulakrishnan Ravi
Dashboard implementation	Gokulakrishnan Ravi
Repository structure and project web page	Gokulakrishnan Ravi
Demonstration video	Gokulakrishnan Ravi
Proposal and report writing	All members
Checkpoint presentations	All members

APPENDIX B PROJECT SCHEDULE

TABLE II
PROJECT SCHEDULE

Period	Milestone
Sep 15–22	Finalize scope, select datasets, assign roles, submit proposal and publish project web page.
Sep 23–Oct 6	Data ingestion, deduplication, temporal splits, and graph construction complete. Baselines trained.
Oct 7–20	First E-GraphSAGE results on both datasets. Checkpoint I: architecture, split logic, baseline comparison.
Oct 21–Nov 3	Model extension implemented; cross-dataset experiments; LLM explanation prototype.
Nov 4–10	Ablations and error analysis; dashboard v1. Checkpoint II and 15-minute presentation.
Nov 11–24	Freeze models; final experiment runs; dashboard v2.
Nov 25–Dec 8	Final presentation; demonstration video; repository cleanup and reproduction instructions.
Dec 9–11	Final report assembly and submission (due 8 PM, Dec 11, no extension).

APPENDIX C ACKNOWLEDGMENT

The authors acknowledge the use of AI tools as reference material to supplement prerequisite knowledge relevant to this course. These tools were consulted to clarify foundational concepts, review background material, and strengthen the team’s collective understanding of topics that fall outside the core coursework covered so far. They served strictly as a supplementary learning resource and did not replace the team’s own analysis, reasoning, or authorship of this proposal.

AI agents will likewise be consulted during code development for reference and learning purposes, to aid the team’s comprehension and implementation of the required techniques. Specifically, they may be used to explain unfamiliar APIs, clarify algorithmic concepts, suggest debugging approaches, and provide general guidance on best practices. All design decisions, implementation choices, and final code will be produced and verified by the team. The role of AI in this project is therefore limited to that of a reference and learning aid — supporting the team’s understanding and skill development rather than directing or authoring the work.